

Ejercicio 16: Juego de las Monedas (Estrategia Óptima)

Enunciado

Considerar un juego con n monedas cuyos valores son $v_1, v_2, \dots, v_n$ (n es par). En cada turno, un jugador elige la moneda del principio o la del final de la fila. El objetivo es determinar el máximo valor que puede obtener el primer jugador, asumiendo que ambos juegan de forma óptima para maximizar sus propias ganancias.

1. Modelado del Problema

Este es un problema clásico de **Minimax** resuelto con Programación Dinámica. La clave es entender que cuando nosotros elegimos una moneda, el oponente jugará de tal forma que nos deje en la peor situación posible en nuestro siguiente turno.

Definiciones

- Sea v el vector de valores de las monedas de 1 a n .
- El estado del juego se define por el intervalo de monedas restantes: $[i, j]$.

2. Estructura de Programación Dinámica

Definición de la Función Óptima

Sea $DP[i][j]$ el **valor máximo** que el jugador cuyo turno es ahora puede garantizarse a partir del intervalo de monedas que va desde la posición i hasta la j .

Relación de Recurrencia

Si tenemos el turno ante el intervalo $[i, j]$, tenemos dos opciones:

1. **Elegir la moneda v_i :**
 - El oponente se queda con el intervalo $[i + 1, j]$.
 - El oponente elegirá entre v_{i+1} o v_j .
 - Como el oponente juega óptimamente para él, nos dejará con el mínimo valor posible para nuestro siguiente movimiento.
 - Por tanto, recibimos: $v_i + \min(DP[i + 2][j], DP[i + 1][j - 1])$.
2. **Elegir la moneda v_j :**
 - El oponente se queda con el intervalo $[i, j - 1]$.
 - De nuevo, el oponente nos dejará en la situación que menos nos convenga.
 - Recibimos: $v_j + \min(DP[i + 1][j - 1], DP[i][j - 2])$.

Ecuación final:

$$DP[i][j] = \max \begin{cases} v_i + \min(DP[i+2][j], DP[i+1][j-1]) \\ v_j + \min(DP[i+1][j-1], DP[i][j-2]) \end{cases}$$

Casos Base

- Si solo hay una moneda: $DP[i][i] = v_i$.
- Si hay dos monedas: $DP[i][i+1] = \max(v_i, v_{i+1})$.

3. Algoritmo y Complejidad

Pseudocódigo

```

Entrada: v[1..n]

// Tabla de n x n inicializada a 0
Para gap de 0 hasta n-1:
    Para i de 1 hasta n - gap:
        j = i + gap

        // x, y, z representan los valores de los saltos dobles
        x = (i+2 <= j) ? DP[i+2][j] : 0
        y = (i+1 <= j-1) ? DP[i+1][j-1] : 0
        z = (i <= j-2) ? DP[i][j-2] : 0

        DP[i][j] = max(v[i] + min(x, y),
                      v[j] + min(y, z))

Retornar DP[1][n]
```

Análisis de Coste

- **Temporal:** $O(n^2)$. Se rellena una matriz triangular de $n \times n$.
- **Espacial:** $O(n^2)$ para almacenar los resultados intermedios.